High Dimensional Data Indexing

INFS 4205/7205 Advanced Techniques for High Dimensional Data

Yadan Luo

School of EECS, The University of Queensland

Recap: MLLM

- (M)LLM are a phenomenal piece of technology for knowledge generation and reasoning. They are pretrained on large amounts of publicly available data.

Llava 1.0

Llava 1.5

Llava-NEXT (1.6)

Meta Chameleon

LLaVA-Onevision

Qwen2-VL

deepseek Janus-Pro-7B

Use Cases

(V)Question-Answering

TextGen: Storytelling

ImageGen: Poster design

Planning

Summarization

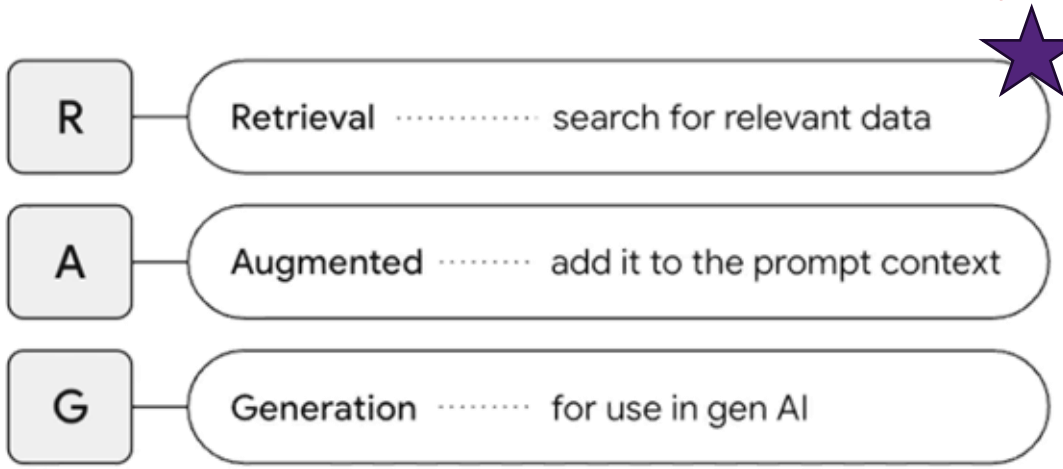
…

Reliable MLLM?

- Without RAG (week 8): (M)LLMs has to be the only source of knowledge (pretraining)

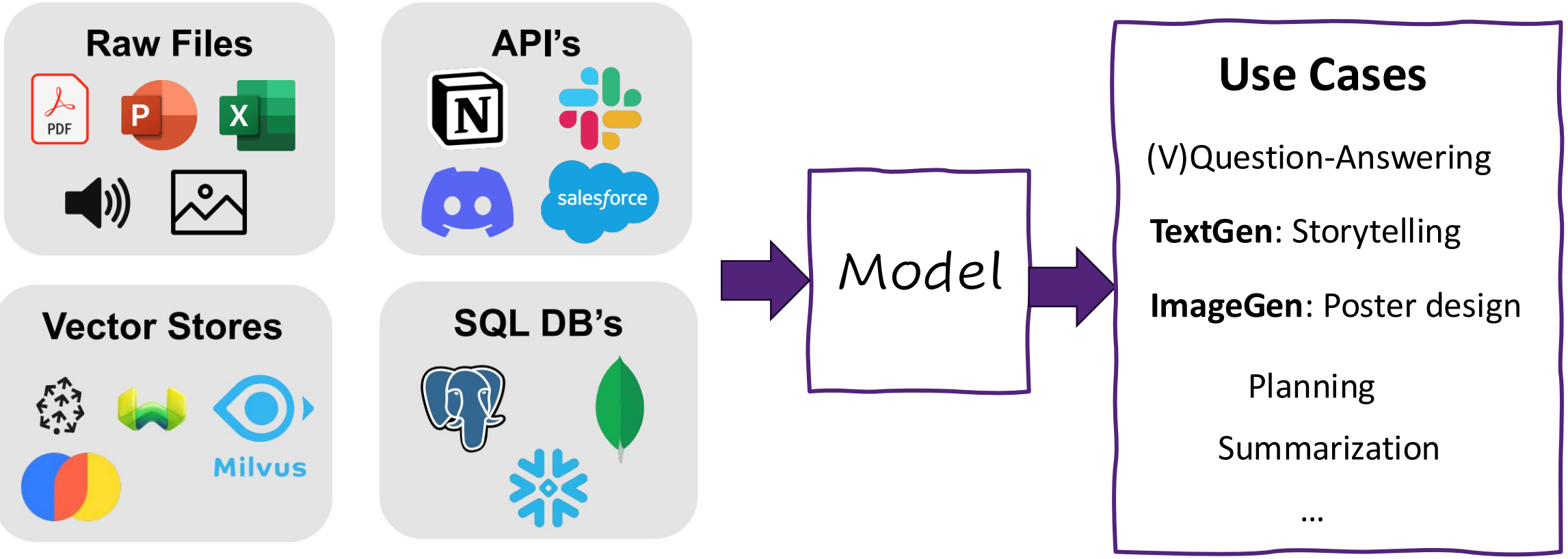
Without RAG	With RAG
Ungrounded hallucination	Grounded on source data (citations)
Old information	Always up to date
No idea what that means	As comprehensive as search
Not my data	My data and/ or public data

Our focus today

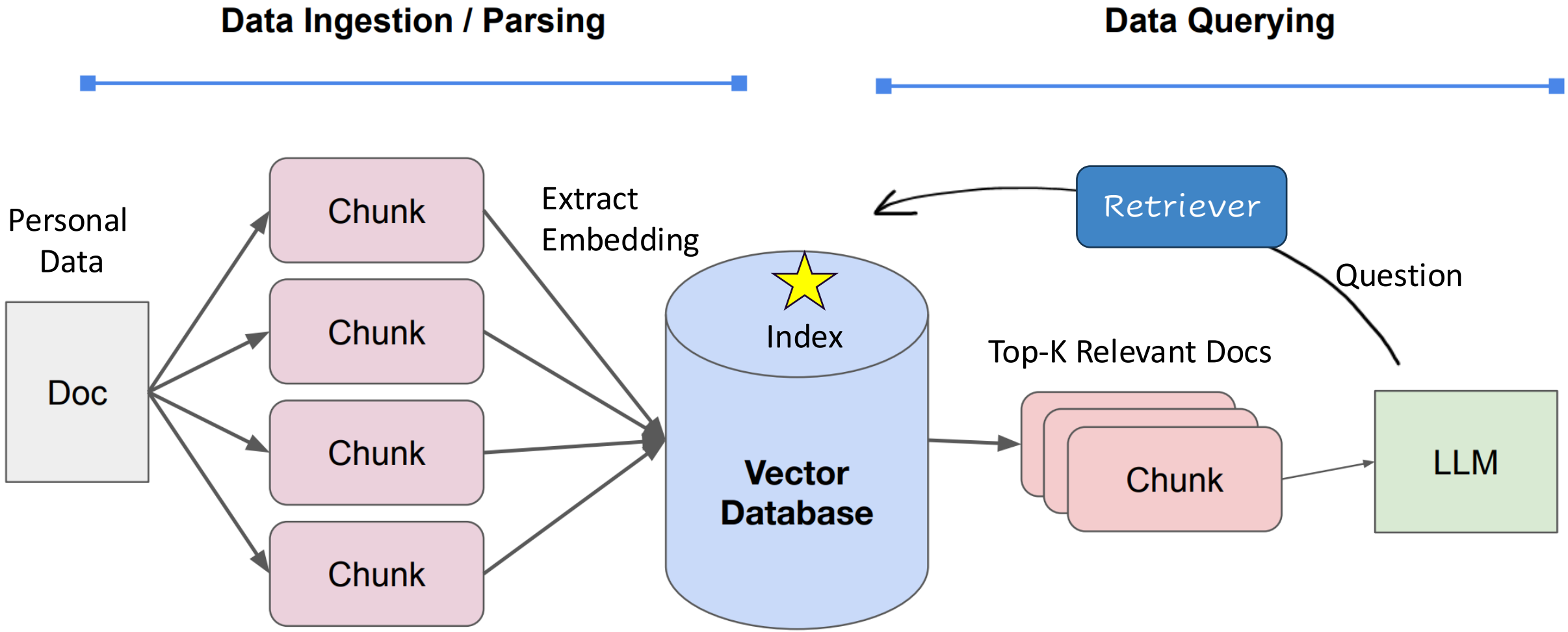


Reliable (M)LLM: From Generalist to Specialist

- How do we best augment (M)LLMs with our own **private data**?

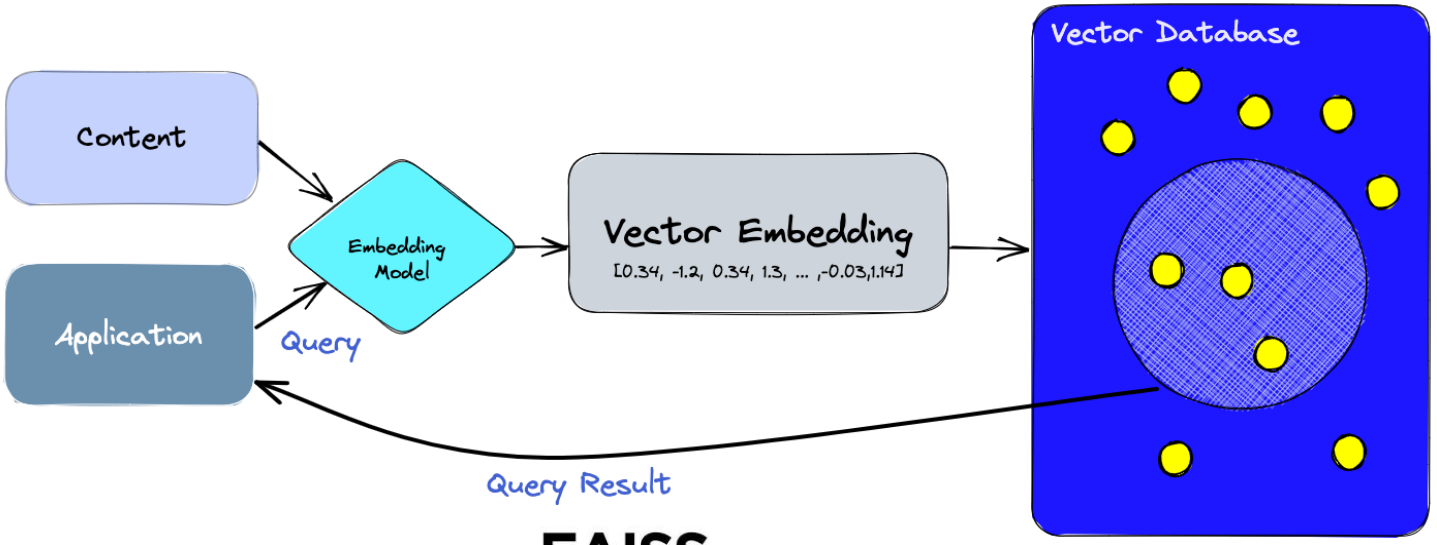


Multimodal RAG Pipeline (Week 8)

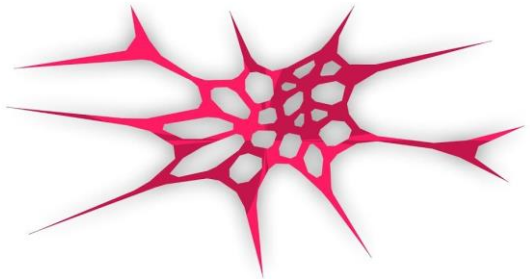


What is Vector DB?

- a specialized database designed to store, manage, and query high-dimensional vector embeddings



FAISS
Scalable Search With Facebook AI



Milvus

chroma

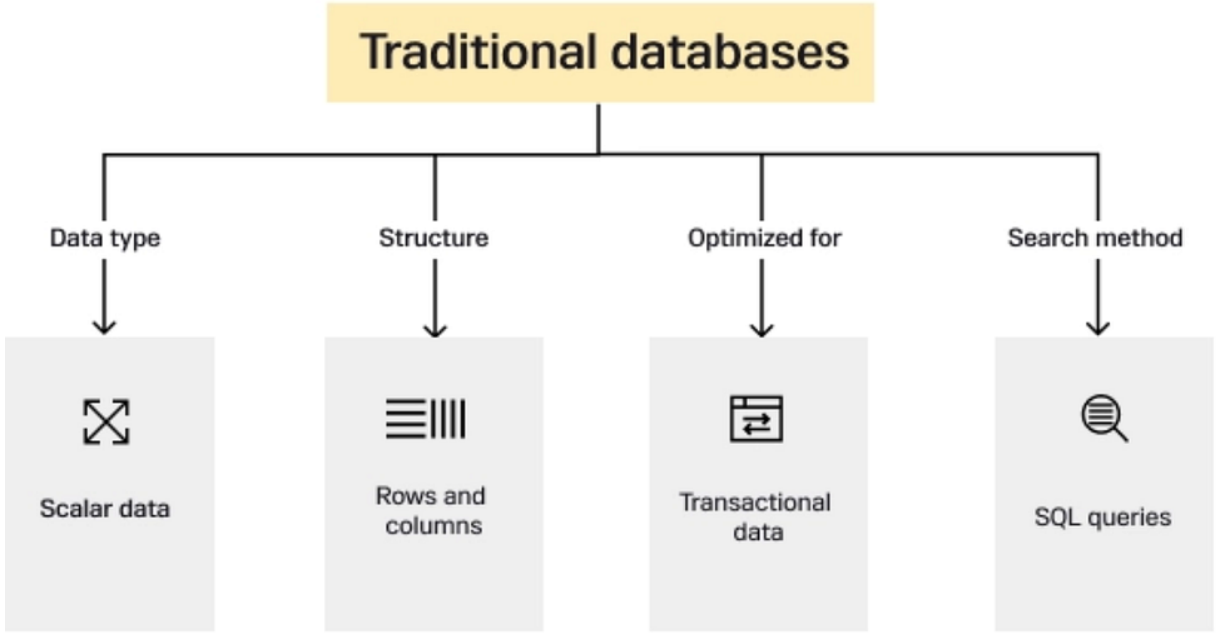
vespa

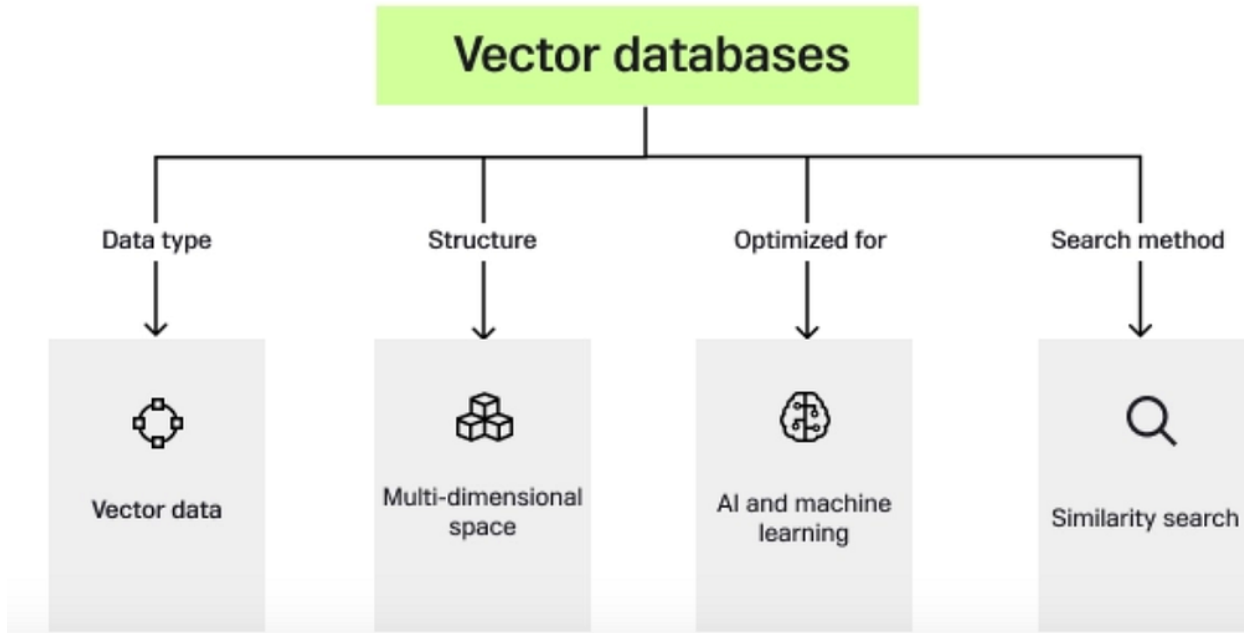
marqo

drant

LanceDB

What is Vector DB?





- Traditional DB: **discrete**, **scalar** data types (numbers and strings) in rows/ columns.
- Vector DB: **high-dim** vector data (embeddings) for similarity search

Why Similarity Search?

Method	Works Well In	Performance Degrades In	Notes
KD-Tree	≤ 10D	≥ 20D	Poor recall, linear scan-like
R-Tree	2D–3D	≥ 5D	Overlapping bounding boxes

- *If we stick to exact kNN search…* (recall in Lec1, as dimension **goes up**, volume grows exponentially, traditional indexing (e.g., KD-trees collapses into O(N) linear scans)

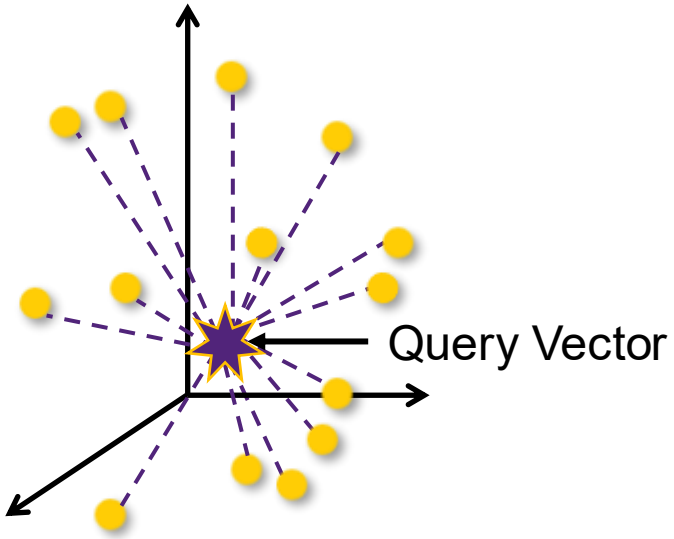
Exact k-Nearest Neighbors	The Math	The Verdict
Requires a full linear scan of the entire dataset.	50,000,000 records × 4096 dimensions = Computational paralysis.	Brute force is the enemy of real-time AI.

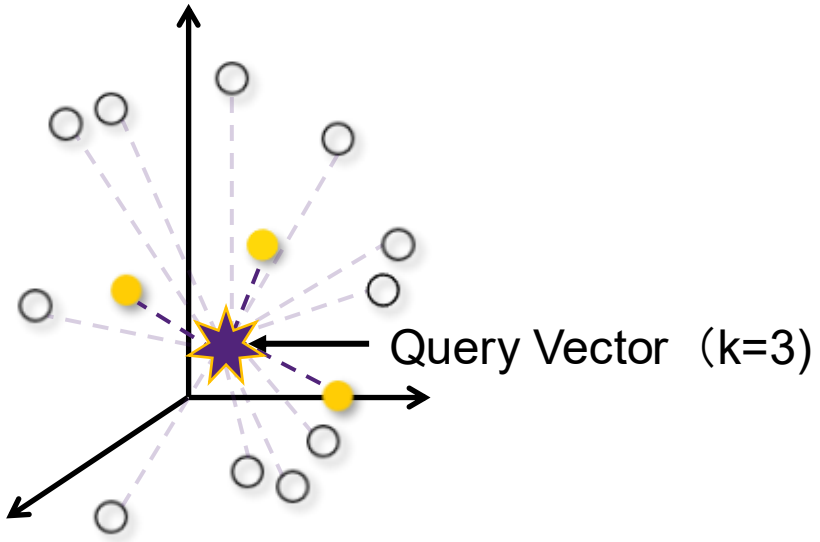
Why Similarity Search?

- *If we stick to exact kNN search…*

  Flat Index (Brute Force**)**

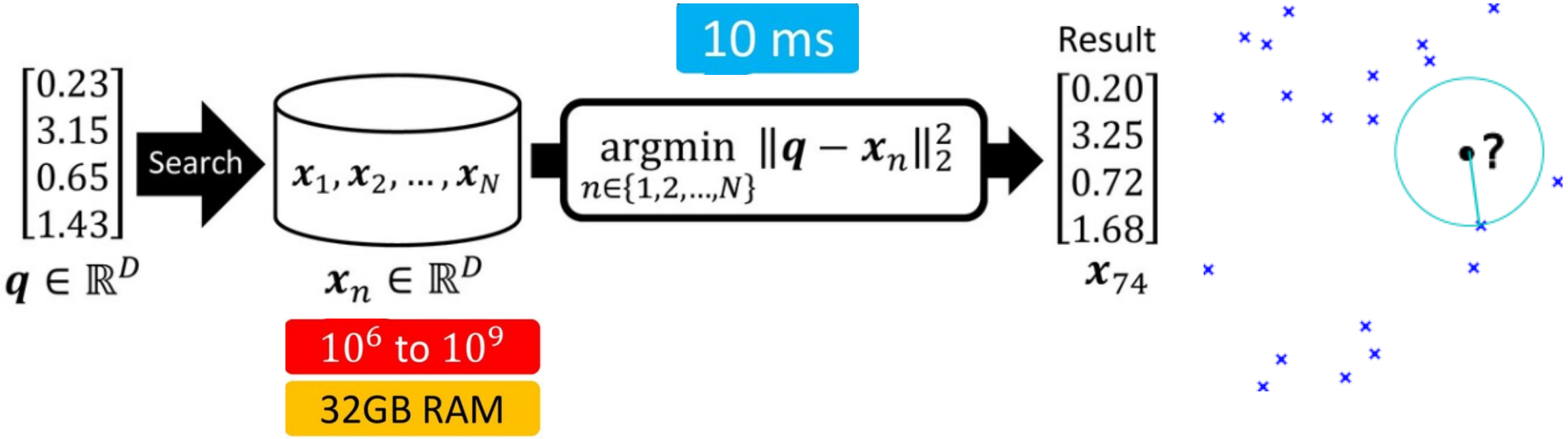
  - Flat indices are a *direct representation* of the vector embedding
  - Deliver the best accuracy of all indexing methods, but notoriously slow
  - Search is *exhaustive:* it's performed from the query vector across *every single vector embedding* and distances are calculated for each pair.





Approximate Nearest Neighbour Search (ANN)

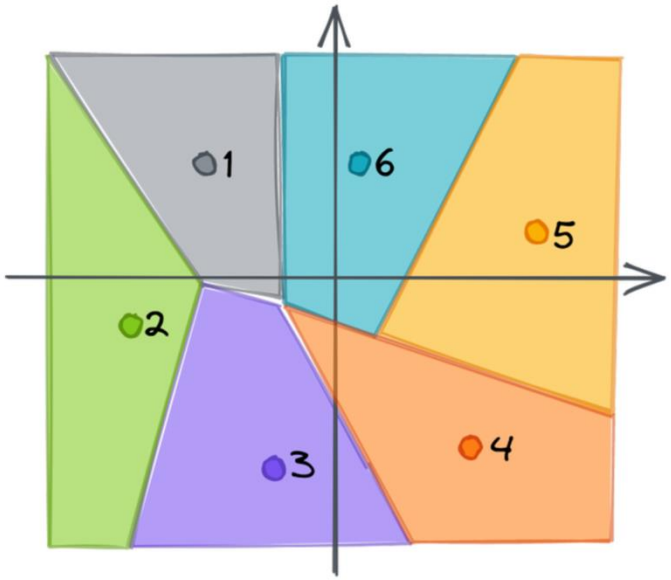
- Allowed to return points, whose distance from the query is at most c times the distance from the query to its nearest points

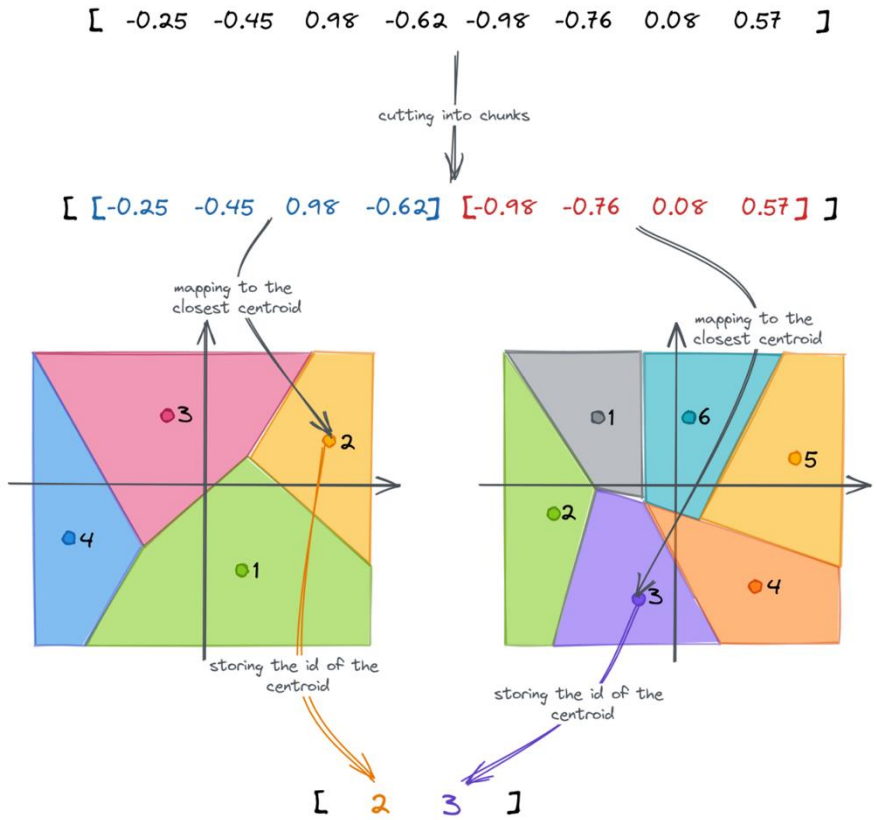


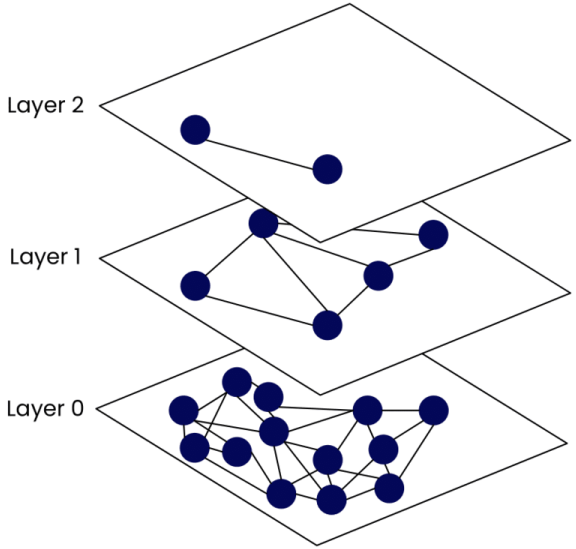
- **Faster Search**: billion-scale data on memory
- **Trade-offs**: runtime, accuracy, memory-consumption

How to achieve ANN?

Inverted files (IVF)







Partitioning

split the whole data into several clusters using K-means

Quantization

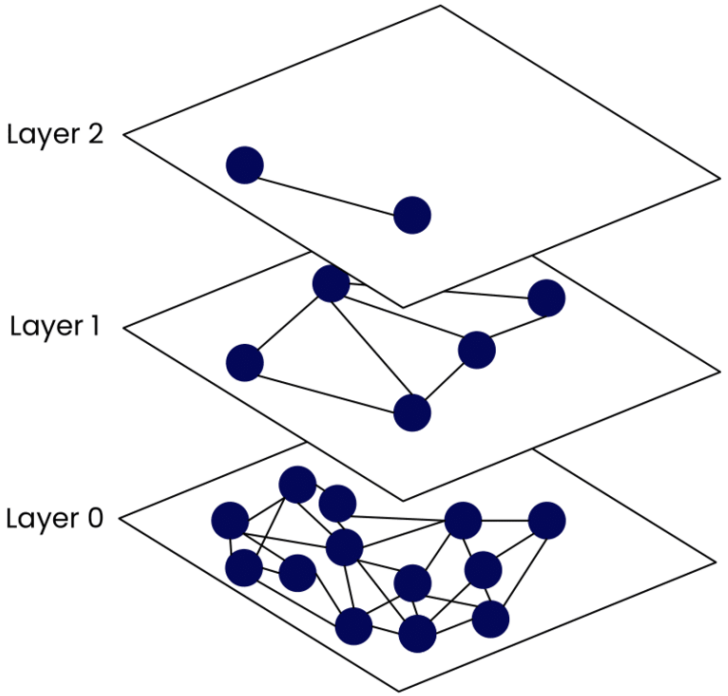
(e.g., Product Quantization) compress the vector

Graphs

(e.g., HNSW) navigate the space

Hierarchical Navigable Small Worlds (HNSW)

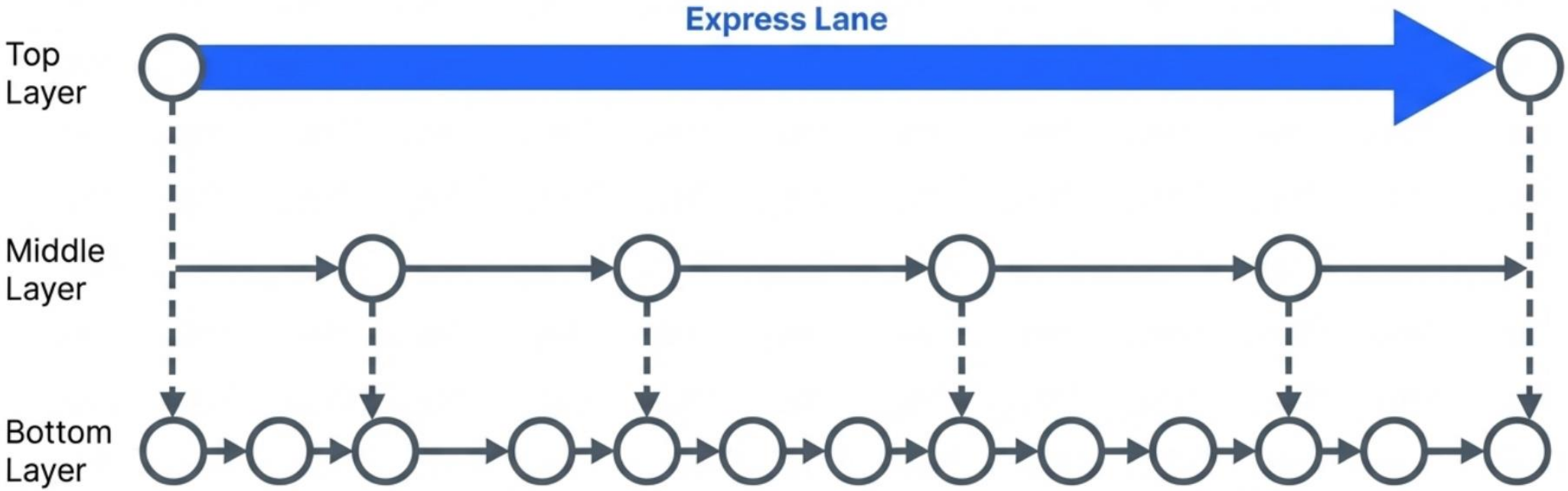
- "proximity" graph indexing; construct a network-like structure
  - **Nodes:** vector embeddings
  - **Edge:** relationship between embeddings (often Euclidean distance).
- Starting at a predefined "entry point", the algorithm **traverses** connected friend vertices until it finds the nearest neighbour to the query vector



Hierarchical Navigable Small Worlds (HNSW)

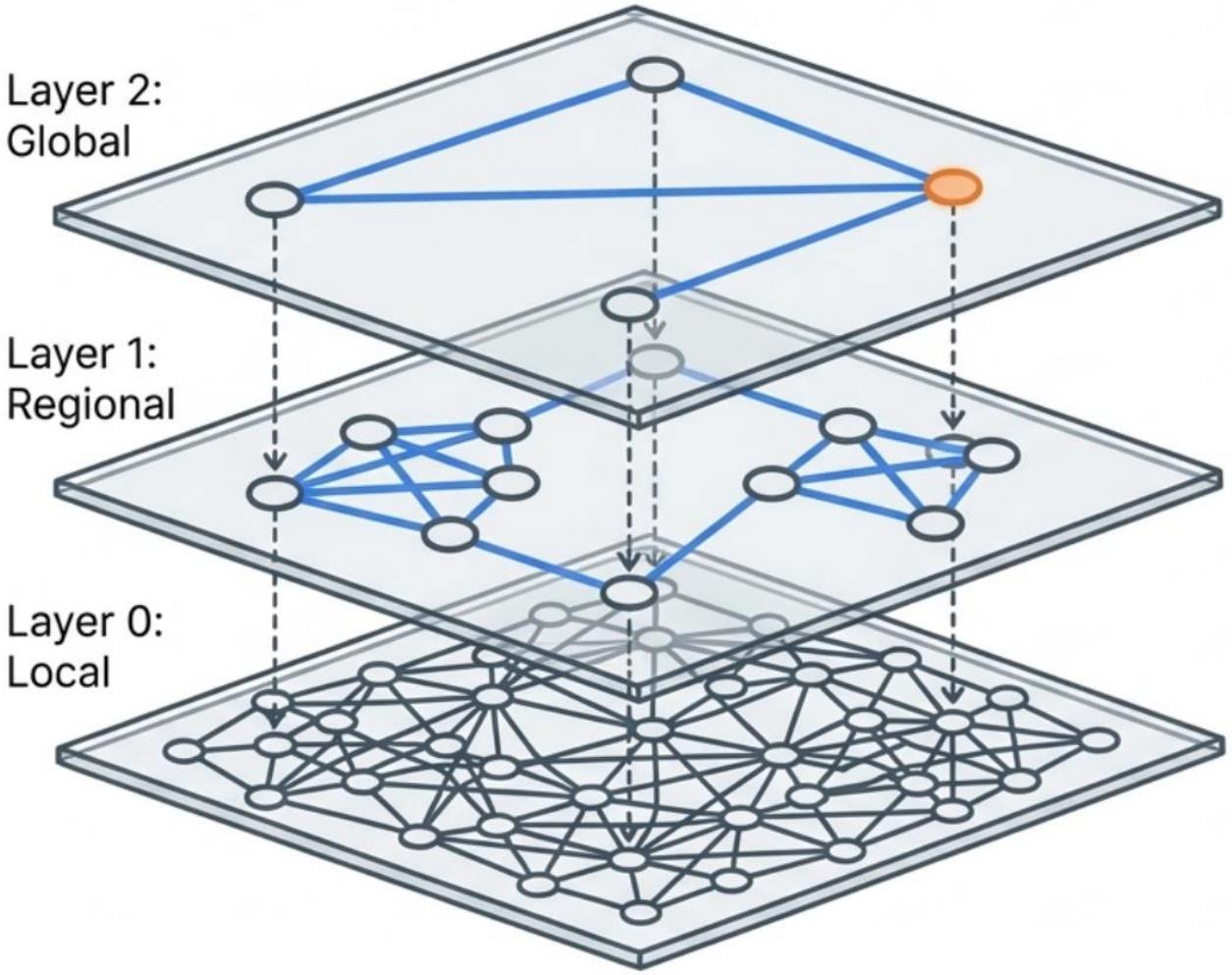
The Challenge: How to search a sorted list in O(log N) time instead of O(N)?

Motivated by 1D Skip List…



The Mechanism: Add hierarchical express lanes for massive long jumps.

Hierarchical Navigable Small Worlds (HNSW)



The Gold Standard

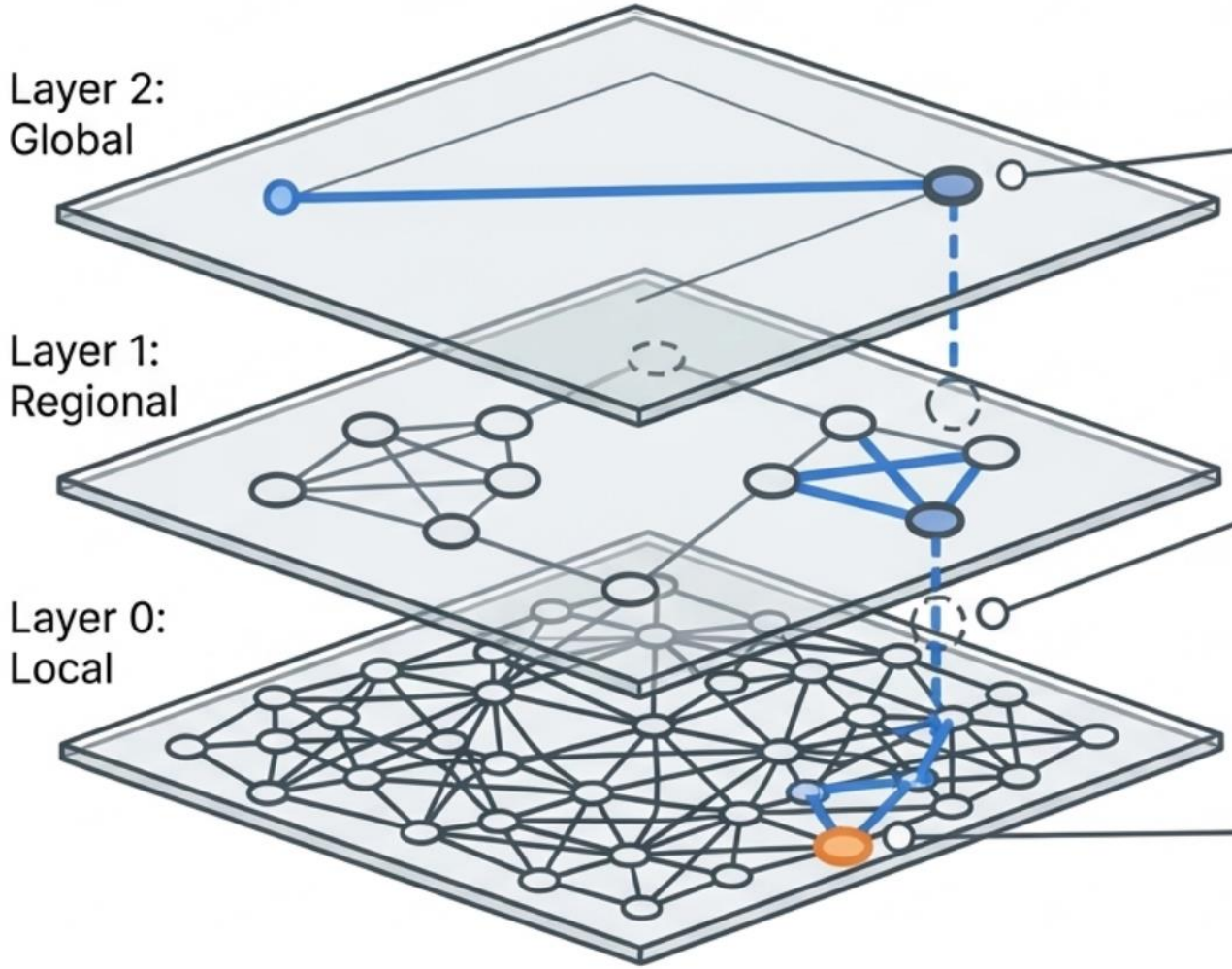
The dominant algorithm for modern Vector Search.

The Architecture

Extends the 1D skip list concept into high-dimensional geometric graphs.

Layer 2	Global navigation (Long jumps)
Layer 1	Regional navigation
Layer 0	Local, exact neighborhood search

Hierarchical Navigable Small Worlds (HNSW)



The Logic

Execute a greedy search at each individual layer.

The Drop

Descend to the next, denser layer only when a local minimum is reached.

The Result

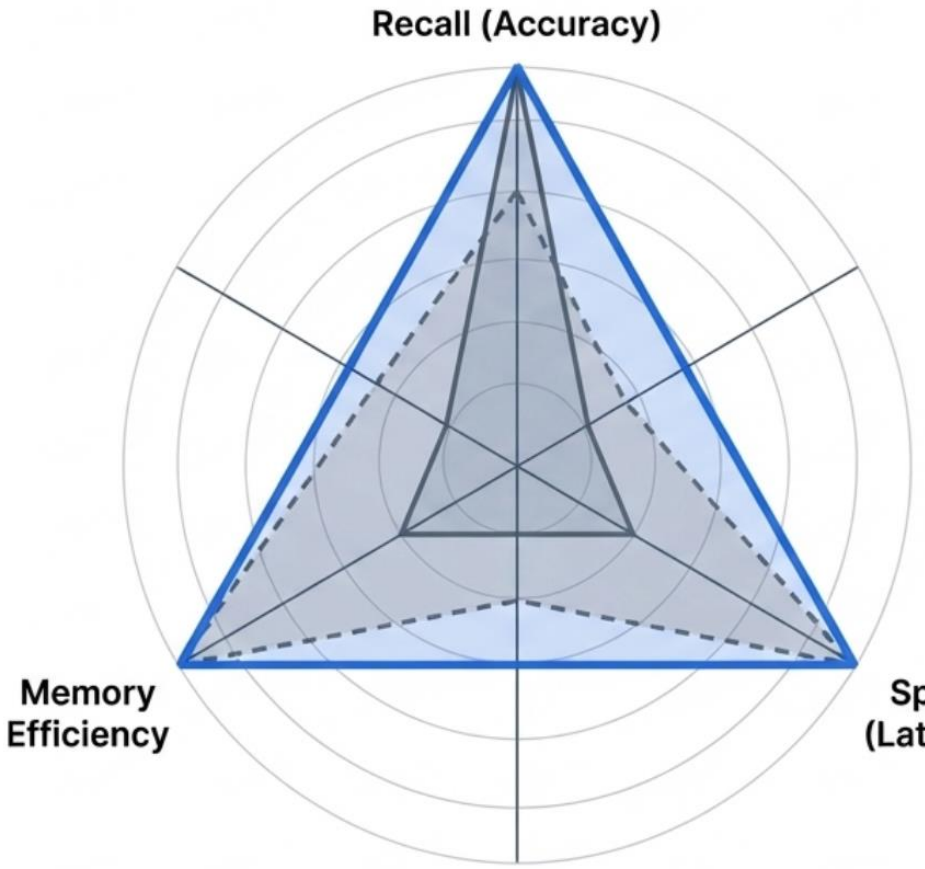
Lightning-fast, $O(\log N)$ traversal.

Hierarchical Navigable Small Worlds (HNSW)

Algorithm Summary: The Trade-off Matrix

Flat (Exact)

Perfect recall, slow speed, high memory overhead.



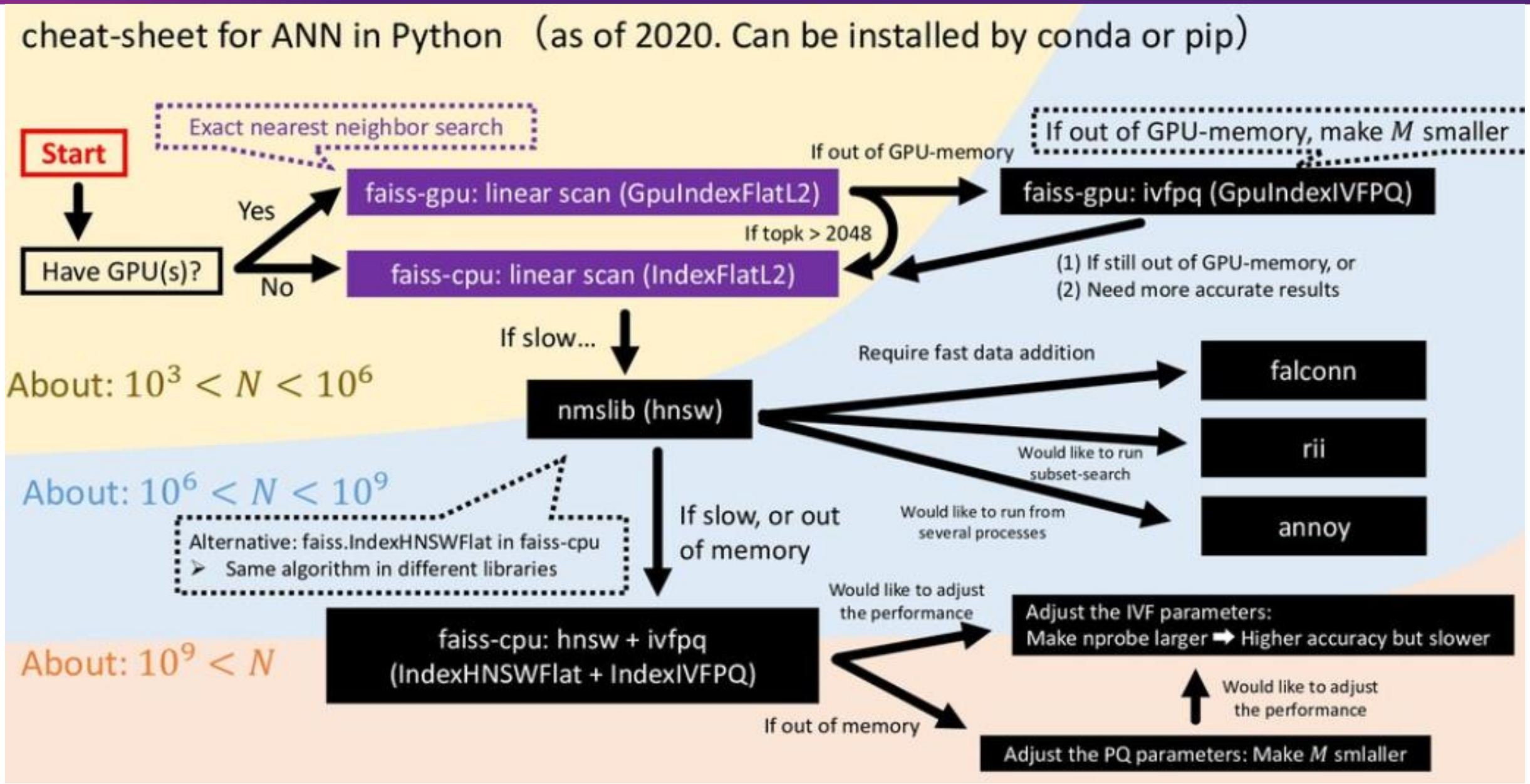
- Pinecone ..... Proprietary composite index
- milvus / zilliz ..... Flat, Annoy, IVF, HNSW/RHNSW (Flat/PQ), DiskANN
- Weaviate ..... Customized HNSW, HNSW (PQ), DiskANN (in progress...)
- drant ..... Customized HNSW
- chroma ..... HNSW
- LanceDB ..... IVF (PQ), DiskANN (in progress...)
- vespa ..... HNSW + BM25 hybrid
- Vald ..... NGT
- elasticsearch ..... Flat (brute force), HNSW
- redis ..... Flat (brute force), HNSW
- pgvector ..... IVF (Flat), IVF (PQ) in progress...

IVF-PQ

Good recall, high speed, ultra-low memory.

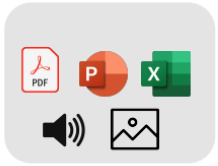
HNSW

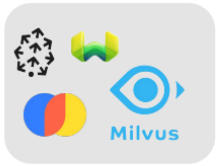
Extremely high recall, ultra-high speed, massive memory requirement.


cheat-sheet for ANN in Python （as of 2020. Can be installed by conda or pip）
Start
Exact nearest neighbor search
Have GPU(s)?
Yes
No
faiss-gpu: linear scan (GpuIndexFlatL2)
faiss-cpu: linear scan (IndexFlatL2)
If out of GPU-memory
If out of GPU-memory, make M smaller
faiss-gpu: ivfpq (GpuIndexIVFPQ)
If topk > 2048
(1) If still out of GPU-memory, or
(2) Need more accurate results
If slow...
About: $10^3 < N < 10^6$
nmslib (hnsw)
Require fast data addition
falconn
Would like to run subset-search
rii
Would like to run from several processes
annoy
About: $10^6 < N < 10^9$
Alternative: faiss.IndexHNSWFlat in faiss-cpu
➢ Same algorithm in different libraries
If slow, or out of memory
About: $10^9 < N$
faiss-cpu: hnsw + ivfpq
(IndexHNSWFlat + IndexIVFPQ)
Would like to adjust the performance
Adjust the IVF parameters:
Make nprobe larger ➡ Higher accuracy but slower
Would like to adjust the performance
If out of memory
Adjust the PQ parameters: Make M smlaller

LlamaIndex: data framework for (M)LLM

- Data Management and Query Engine for your (M)LLM application
- Offers components across the data lifecycle: ingest, index, and query over data





Data Ingestion
(LlamaHub 🦙)

- Connect your existing data sources and data formats (API's, PDF's, docs, SQL, etc.)

Data Structures

- Store and index your data for different use cases. Integrate with different db's (vector db, graph db, kv db)

Queries

- Retrieve and query over data
- Includes: QA, Summarization, Agents, and more

Data Connectors: powered by LlamaHub

- Easily ingest any kind of data, from anywhere
- Growing support for multimodal documents (e.g. with inline images)

```
from llama_index import download_loader
import os

NotionPageReader = download_loader('NotionPageReader')

integration_token = os.getenv("NOTION_INTEGRATION_TOKEN")
page_ids = ["<page_id>"]
reader = NotionPageReader(integration_token=integration_token)
documents = reader.load_data(page_ids=page_ids)
```

<10 lines of code to ingest from Notion

Demo

PDF RAG — Index & Embedding Benchmark

Compare FAISS indexing strategies and embedding models on your PDF.

PAGES	CHUNKS	FIGURES	AVG CHUNK LEN
116	175	151	1006

Document | Index Comparison | Embedding Comparison

Parsed Document

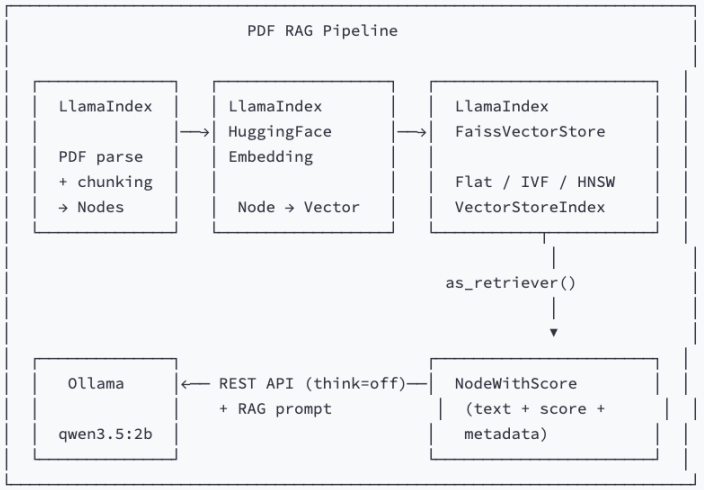
Text Chunks

Chunk 1 · page 1 · 364 chars

107 recipes
all with photos
Chocolate Lover's
Dream Cake, page 83
Celebrating Be/tt y's
classic dishes and
new favorites!
all-time
/ninelining/zerolining
YEARS/spacelininggOF
BETTY/spacelining
CROCKER!
Display until October
/onelining/fourlining/commalining/spacelining/twolining/zerolining/onelining/onelining
V olume /twolining/onelining, Number /fiveliningA

Chunk 2 · page 2 · 0 chars

Extracted Figures



Library	Role in this demo
LlamaIndex — `SimpleDirectoryReader` + `SentenceSplitter`	PDF → Nodes (text chunks with metadata)
LlamaIndex — `HuggingFaceEmbedding`	Embed nodes (wraps sentence-transformers)
LlamaIndex — `FaissVectorStore`	Wraps a raw FAISS index for add / query / persist
LlamaIndex — `VectorStoreIndex` + `StorageContext`	Connects vector store + docstore; provides `as_retriever()`
FAISS (`faiss-cpu`)	Underlying index engine (Flat / IVF / HNSW) — accessed **through** `FaissVectorStore`
Ollama REST API (`think=false`)	Assemble RAG prompt, call local LLM — thinking disabled for speed

Why FaissVectorStore instead of raw FAISS? — It integrates FAISS into LlamaIndex's storage/retrieval pipeline: `StorageContext` manages the vector store + docstore, and `VectorStoreIndex.as_retriever()` handles query embedding and result mapping automatically. You still control the raw FAISS index type and params.

Demo

Step 1 — Parse PDF → Markdown → Chunk (pymupdf4llm)

```python
import pymupdf4llm
from llama_index.core import Document
from llama_index.core.node_parser import SentenceSplitter

# pymupdf4llm preserves tables, headers, multi-column layout as Markdown
pages = pymupdf4llm.to_markdown("data/paper.pdf", page_chunks=True)
docs = [
    Document(text=p["text"], metadata={"page_label": str(p["metadata"]["page_number"])})
    for p in pages
]
splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)
nodes = splitter.get_nodes_from_documents(docs)
```

Demo

Step 2 — Embed nodes (HuggingFaceEmbedding)

```
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embed_model = HuggingFaceEmbedding(model_name="all-MiniLM-L6-v2")
for node in nodes:
    node.embedding = embed_model.get_text_embedding(node.get_content())
```

Demo

Step 3 — Create FaissVectorStore → VectorStoreIndex

```
import faiss
from llama_index.vector_stores.faiss import FaissVectorStore
from llama_index.core import VectorStoreIndex, StorageContext

# Create raw FAISS index (Flat, IVF, or HNSW)
faiss_index = faiss.IndexFlatL2(384)

# Wrap in LlamaIndex FaissVectorStore
vector_store = FaissVectorStore(faiss_index=faiss_index)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# Build VectorStoreIndex — adds pre-embedded nodes to FAISS
index = VectorStoreIndex(
    nodes,
    storage_context=storage_context,
    embed_model=embed_model,
)
```

Demo

Step 4 — Retrieve (embeds query + searches FAISS internally)

```
retriever = index.as_retriever(similarity_top_k=5)
results = retriever.retrieve("What is the main contribution?")

for r in results:
    print(r.score, r.node.get_content()[:200])
```

Similarity scores

Retrieved chunk

Demo

Step 5 — RAG prompt → LLM (Ollama REST API, think=false)

```
import requests

context = "\n".join([r.node.get_content() for r in results])
resp = requests.post("http://localhost:11434/api/chat", json={
    "model": "qwen3.5:2b",
    "messages": [
        {"role": "system", "content": "Answer using ONLY the provided context."},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: ..."},
    ],
    "stream": False,
    "think": False,           # ← disables qwen3.5 chain-of-thought (~10x faster)
    "options": {"temperature": 0, "num_predict": 1024},
})
answer = resp.json()["message"]["content"]
```

System prompt

Response length

Low temp: faithful

High temp: creative

Demo

Flat (Exact Search)

Brute-force L2 distance against *every* vector. 100% recall guaranteed. Used as ground-truth baseline.

```
fi = faiss.IndexFlatL2(dim)
store = FaissVectorStore(faiss_index=fi)  # wrap
```

Pros: exact, simple, no params

Cons: O(N) — slow at 1M+ vectors

Demo

IVF (Inverted File)

K-means clusters vectors into `nlist` buckets. At query time, only searches `nprobe` nearest clusters.

```
quantizer = faiss.IndexFlatL2(dim)
fi = faiss.IndexIVFFlat(quantizer, dim, nlist=16)  How many clusters to build
fi.train(embeddings)                               # k-means training required
fi.nprobe = 4     How many clusters to check
store = FaissVectorStore(faiss_index=fi) # wrap trained index
```

Pros: fast, tunable `nprobe`

Cons: needs training, can miss clusters

> `nprobe=1` fastest, lowest recall. `nprobe=nlist` = brute-force.

Demo

HNSW (Hierarchical Navigable Small World)

Multi-layer proximity graph. Each vector is a node with `M` edges to nearby neighbors. Search greedily walks the graph.

How many edges; affect memory consumption & recall

```
fi = faiss.IndexHNSWFlat(dim, M=16)
fi.hnsw.efConstruction = 200          # build quality
fi.hnsw.efSearch = 64                 # query-time beam width
store = FaissVectorStore(faiss_index=fi) # wrap
```

Seach width for build & query

Pros: best recall/speed tradeoff, no training

Cons: more memory (graph edges), no deletion

> `M` = connectivity, `efSearch` = beam width. Both higher → better recall, slower.

Demo

◎ PDF RAG Demo

LlamaIndex · LangChain · FAISS · Ollama

PDF source

Local (data/) Upload

Select PDF

gm_alltimebest_vol21no5.pdf

gm_alltimebest_vol21no5.pdf loaded

Chunking

Chunk size

256

Overlap

50

Top-K

12

Embedding models

all-MiniLM-L6-v2 ×

all-mpnet-base-v2 ×

FAISS Index Comparison

Flat (exact) vs IVF (inverted file) vs HNSW (graph-based) — retrieval **and** RAG answer impact.

Embedding model

all-MiniLM-L6-v2

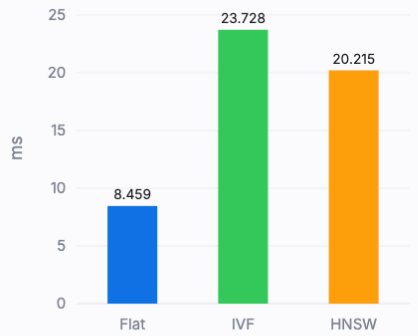
Search query

how to cook deviled eggs?

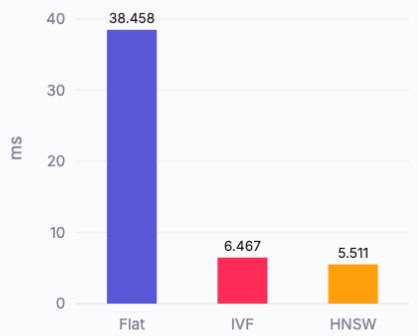
Embedded 319 chunks (384-d) in **1.77 s** with `all-MiniLM-L6-v2`

Performance

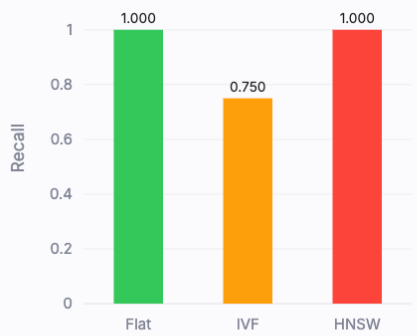
Build Time



Query Time



Recall@12



Retrieved Chunks

Flat (Exact)

› #1 · score 0.6254 · p.16

› #2 · score 0.7568 · p.16

IVF (Approx)

› #1 · score 0.6254 · p.16

› #2 · score 0.7568 · p.16

HNSW (Graph)

› #1 · score 0.6254 · p.16

› #2 · score 0.7568 · p.16

Demo

LLM Response Time

Flat (Exact)

27.0s · 6507 ctx chars · nodes ['86543628', '2384620d', '4ceaecb3', '4ed63bc3', '76ee2475']

Based on the provided context, there are two distinct recipes for "Deviled Eggs" found in the document:

1. Shrimp Deviled Eggs

- **Prep Time:** 30 Minutes
- **Cooking Time:** 30 Minutes
- **Ingredients:**
  - 6 hard-cooked eggs
  - 2 medium green onions, thinly sliced
  - ¼ cup reduced-fat mayonnaise or salad dressing
  - 1 tablespoon white vinegar
  - ¼ teaspoon salt
  - ¼ teaspoon red pepper sauce
  - ½ cup coarsely chopped cooked salad shrimp (thawed if frozen)

IVF (Approx)

8.2s · 6630 ctx chars · nodes ['2384620d', '4ceaecb3', '4ed63bc3', '76ee2475', 'd5ecd2db']

Based on the provided context, there is no recipe for "devil eggs."
The document contains two recipes that use the term "devil's food" (a cake mix), but neither is for eggs:

1. **Gluten Free Zucchini-Devil's Food Snack Cake** (Passage 2): This recipe uses a cake mix, zucchini, and eggs to make a cake, not deviled eggs.
2. **Pull-Apart Turtle Cupcakes** (Passage 3): This recipe uses a cake mix, frosting, and candies to make cupcakes shaped like turtles, not deviled eggs.

The only recipe that actually makes "devil eggs" is the **Deviled Eggs** recipe in **Passage 1**, which is not included in the provided text.

HNSW (Graph)

Same chunks as Flat (Exact) — skipped LLM call

Based on the provided context, there are two distinct recipes for "Deviled Eggs" found in the document:

1. Shrimp Deviled Eggs

- **Prep Time:** 30 Minutes
- **Cooking Time:** 30 Minutes
- **Ingredients:**
  - 6 hard-cooked eggs
  - 2 medium green onions, thinly sliced
  - ¼ cup reduced-fat mayonnaise or salad dressing
  - 1 tablespoon white vinegar
  - ¼ teaspoon salt
  - ¼ teaspoon red pepper sauce
  - ½ cup coarsely chopped cooked salad shrimp (thawed if frozen)
  - 1 tablespoon cocktail sauce

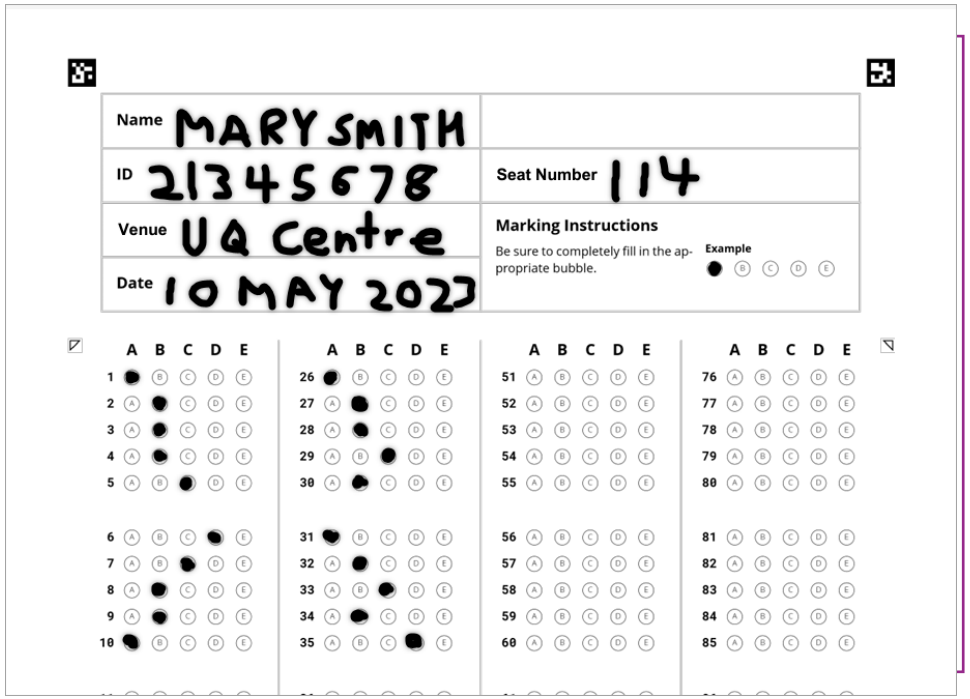
What we covered today…

After today's class, you should be able to

- understand the **RAG** framework using private, up-to-date data to prevent hallucinations.
- distinguish between traditional scalar databases (SQL) and multi-dimensional Vector Databases.
- why Approximate Nearest Neighbour (ANN) search is essential for industrial scale.
- **Core Indexing:** Space clustering (**IVF**) and Graph-based routing (**HNSW**).
- **Data Pipelines:** Ingesting, chunking, and indexing documents using **LlamaIndex**.

Next Week: Happy Semester Break ☺

Gradescope Bubble Sheet tips



1. Your exam will use a Gradescope Bubble Sheet
2. Enter your **FIRST NAME** & **LAST NAME** in block capitals
3. Enter your eight digit UQ student **ID**
4. For central exams enter the venue and your seat number
5. Enter the date of your exam
6. Fill in the bubbles using a **2B pencil**
7. To change an answer use an **eraser**

THANKS!

Do you have any questions?

uqyluo@uq.edu.au
The University of Queensland